# FILE SHARING SYSTEM (PRODUCTION EXECUTION DOCUMENT)

**Naiyo24 Private Limited – Internal Engineering Document**

---

# 1. PROJECT OVERVIEW

This document defines the **complete architecture, system design, development plan, and execution guidelines** for building a **production-grade file sharing system (WeTransfer-style)**.

The system must support:

- Large file uploads (up to 5GB+)
- Secure sharing via unique links
- Expiry-based access control
- Resume upload & download
- High concurrent usage

This is a **production system**, not a demo. All implementations must follow **scalable and maintainable architecture principles**.

---

# 2. OBJECTIVES

- Build a **robust and scalable backend**
- Enable **chunk-based upload system**
- Ensure **secure file access & sharing**
- Provide **clean Flutter user experience**
- Design system for **1L+ users scalability**

---

# 3. TECH STACK

### Frontend

- Flutter
- Riverpod (State Management)

- Dio (Networking)

### Backend

- FastAPI
- JWT Authentication
- Background Tasks (Celery / FastAPI BackgroundTasks)

### Database

- PostgreSQL

### Cache

- Redis

### Storage

- AWS S3 / MinIO

### Infrastructure

- Docker + Docker Compose
- Nginx
- CDN (Cloudflare / AWS CloudFront)

---

# 4. SYSTEM FLOW

1. User selects file in Flutter app
2. Backend creates upload session
3. File split into chunks
4. Chunks uploaded sequentially
5. Stored in S3
6. Upload finalized
7. Share link generated
8. User shares link
9. Receiver clicks link
10. Backend validates expiry & access
11. File served via CDN

---

# 5. BACKEND ARCHITECTURE (FASTAPI)

## COMPLETE BACKEND FILE STRUCTURE

```
backend/
│
├── app/
│   ├── main.py                     # Entry point
│
│   ├── core/                       # Core configurations
│   │   ├── config.py
│   │   ├── security.py
│   │   ├── database.py
│   │   ├── dependencies.py
│
│   ├── api/
│   │   ├── routes/
│   │   │   ├── upload.py
│   │   │   ├── share.py
│   │   │   ├── download.py
│   │   │   ├── auth.py
│   │   │
│   │   ├── router.py
│
│   ├── models/
│   │   ├── user.py
│   │   ├── file.py
│   │   ├── share.py
│   │   ├── download.py
│
│   ├── schemas/
│   │   ├── user_schema.py
│   │   ├── file_schema.py
│   │   ├── share_schema.py
│
│   ├── services/
│   │   ├── upload_service.py
│   │   ├── share_service.py
│   │   ├── download_service.py
│   │   ├── storage_service.py
│
│   ├── utils/
│   │   ├── chunk_handler.py
│   │   ├── file_utils.py
│   │   ├── logger.py
│
│   ├── workers/
│   │   ├── celery_worker.py
│   │   ├── tasks.py
│
├── migrations/                     # Alembic migrations
├── tests/                          # Unit tests
│   ├── test_upload.py
│   ├── test_share.py
```

```
│
├── Dockerfile
├── docker-compose.yml
├── requirements.txt
├── .env
```

---

# 6. FRONTEND ARCHITECTURE (FLUTTER)

## COMPLETE FLUTTER FILE STRUCTURE

```
flutter_app/
│
├── lib/
│   ├── main.dart
│   ├── app.dart
│
│   ├── core/
│   │   ├── constants/
│   │   │   ├── api_constants.dart
│   │   │   ├── app_constants.dart
│   │   │
│   │   ├── theme/
│   │   │   ├── app_theme.dart
│   │   │
│   │   ├── utils/
│   │   │   ├── logger.dart
│   │   │   ├── file_utils.dart
│   │   │   ├── validators.dart
│   │   │
│   │   ├── network/
│   │   │   ├── dio_client.dart
│   │   │   ├── interceptors.dart
│
│   ├── services/
│   │   ├── api_service.dart
│   │   ├── upload_service.dart
│   │   ├── storage_service.dart
│
│   ├── providers/
│   │   ├── app_provider.dart
│   │   ├── upload_provider.dart
│   │   ├── share_provider.dart
│
│   ├── features/
│
│   │   ├── upload/
│   │   │   ├── data/
│   │   │   │   ├── models/
│   │   │   │   │   ├── file_model.dart
│   │   │   │   │   ├── upload_session_model.dart
```

```
│   │   │   │
│   │   │   │   ├── datasources/
│   │   │   │   │   ├── upload_remote_datasource.dart
│   │   │   │   │
│   │   │   │   ├── repositories/
│   │   │   │       ├── upload_repository_impl.dart
│   │   │   │
│   │   │   ├── domain/
│   │   │   │   ├── entities/
│   │   │   │   │   ├── file_entity.dart
│   │   │   │   │
│   │   │   │   ├── repositories/
│   │   │   │   │   ├── upload_repository.dart
│   │   │   │   │
│   │   │   │   ├── usecases/
│   │   │   │       ├── start_upload.dart
│   │   │   │       ├── upload_chunk.dart
│   │   │   │       ├── complete_upload.dart
│   │   │   │
│   │   │   ├── presentation/
│   │   │   │   ├── screens/
│   │   │   │   │   ├── upload_screen.dart
│   │   │   │   │
│   │   │   │   ├── widgets/
│   │   │   │   │   ├── file_picker_widget.dart
│   │   │   │   │   ├── progress_bar.dart
│   │   │   │   │
│   │   │   │   ├── controllers/
│   │   │   │       ├── upload_controller.dart
│   │   │
│   │   ├── share/
│   │   │   ├── data/
│   │   │   ├── domain/
│   │   │   ├── presentation/
│   │   │   │   ├── screens/
│   │   │   │       ├── share_screen.dart
│   │
│   │   ├── download/
│   │   │   ├── data/
│   │   │   ├── domain/
│   │   │   ├── presentation/
│   │   │   │   ├── screens/
│   │   │   │       ├── download_screen.dart
│
│   ├── routes/
│   │   ├── app_routes.dart
│   │   ├── route_generator.dart
│
│   ├── config/
│   │   ├── env.dart
│
├── assets/
│   ├── images/
│   ├── icons/
│
├── pubspec.yaml
```

---

# 7. CORE MODULES

### Upload Service

- Chunk upload (mandatory)
- Resume upload
- Multi-file support

### Share Service

- Unique secure link
- Password protection (future)

### Expiry System

- Auto expiry
- Access validation

### Download Service

- Resume support
- CDN delivery

---

# 8. DATABASE DESIGN

### users

- id, email, password, created_at

### files

- id, filename, size, storage_url, uploaded_by

### shares

- id, file_id, token, expiry_time, password, limit

### downloads

- id, file_id, downloaded_at, ip_address

---

# 9. REDIS USAGE

- Upload session tracking
- Chunk validation
- Share link caching
- Rate limiting

---

# 10. STORAGE STRUCTURE

```
bucket/
└── uploads/
    └── user_id/
        └── file_id/
            └── final_file
```

---

# 11. DOCKER SETUP

Services:

- Backend
- PostgreSQL
- Redis
- Nginx

---

# 12. DEVELOPMENT PHASES

### Phase 1

- Upload + Share

### Phase 2

- Expiry + Tracking

**Phase 3**

- Security + Resume

---

# 13. TEAM RESPONSIBILITIES

**Frontend**

- UI + Upload flow

**Backend**

- APIs + Logic + Security

**QA**

- Testing + Load validation

---

# 14. SECURITY GUIDELINES

- Signed URLs
- Request validation
- Rate limiting
- Secure storage

---

# 15. PERFORMANCE REQUIREMENTS

- Upload success rate > 98%
- Fast download via CDN
- Low API latency

---

# 16. DEPLOYMENT PROCESS

1. docker compose up -d --build

2. Run migrations
3. Configure Nginx
4. Attach domain
5. Enable CDN

---

# 17. ADVANCED IMPLEMENTATION SUPPORT

### Backend Support

- Complete FastAPI code (ready-to-run)
- Chunk upload + S3 + Redis integration

### Frontend Support

- Full Flutter upload implementation
- Progress + retry system

### DevOps Support

- Domain setup
- SSL (HTTPS)
- CI/CD pipeline
- Production Docker optimization

---

# 18. FUTURE SCOPE

- Email sharing
- Team collaboration
- File preview
- Analytics

---

# 19. FINAL INSTRUCTION

This must be built as a **production system**.

All developers must:

- Follow architecture strictly
- Maintain clean code
- Build for scale (1L+ users)

Failure to follow this will result in rework.

---

**Reference Base Document:**

---

# END OF DOCUMENT